

Scalable Academic Policy QA System

CS-404 Big Data Analytics

Project Report

Group Members:	CMS ID
Muhammad Tayyab Rizwan	463804
Ali Asghar Khan Lodhi	478734
Mudassir Ahmed Shaikh	464473

Table of Contents

1. System Design **Error! Bookmark not defined.**

 1.1 Architectural Overview..... **Error! Bookmark not defined.**

 1.2 Key Components **Error! Bookmark not defined.**

 1.2.1 Data Ingestion (ingestion/) **Error! Bookmark not defined.**

 1.2.2 Indexing Layer (indexing/) **Error! Bookmark not defined.**

 1.2.3 Unified Retriever (retriever.py) **Error! Bookmark not defined.**

 1.2.4 Answer Generation (answer/) **Error! Bookmark not defined.**

 1.2.5 Output Interface (interface/app.py)..... **Error! Bookmark not defined.**

2. Algorithm Explanation **Error! Bookmark not defined.**

 2.1 TF-IDF: Baseline Exact Retrieval..... **Error! Bookmark not defined.**

 2.2 MinHash LSH: Approximate Set Similarity..... **Error! Bookmark not defined.**

 2.3 SimHash: Approximate Fingerprinting **Error! Bookmark not defined.**

 2.4 PageRank: Competitive Extension **Error! Bookmark not defined.**

3. Experimental Results **Error! Bookmark not defined.**

 3.1 Exact vs. Approximate Retrieval **Error! Bookmark not defined.**

 3.2 Parameter Sensitivity Analysis..... **Error! Bookmark not defined.**

 3.3 Scalability Test **Error! Bookmark not defined.**

 3.4 Qualitative Evaluation of Answer Correctness..... 11

4. Tradeoff Analysis **Error! Bookmark not defined.**

 4.1 Accuracy vs. Retrieval Speed..... **Error! Bookmark not defined.**

 4.2 Memory Footprint vs. Precision **Error! Bookmark not defined.**

 4.3 Keyword Matching vs. Structural Similarity **Error! Bookmark not defined.**

 4.4 Structural Boosting (PageRank) vs. Semantic Relevance **Error! Bookmark not defined.**

1. System Design

This section presents the end-to-end architecture of the Scalable Academic Policy Question Answering (QA) System, designed to ingest, index, and query the NUST Undergraduate Student Handbook. The system integrates a parallel retrieval architecture, an LLM-powered answer synthesis layer, and a professional multi-tab Streamlit dashboard for interactive use and rigorous experimentation.

1.1 Architectural Overview

The pipeline is structured into five distinct layers, each encapsulating a specific stage of the QA workflow. Data flows from raw PDF ingestion through structured chunking, parallel indexing, unified retrieval, answer generation, and finally to an interactive output interface. The architecture is illustrated in Figure 1 below.

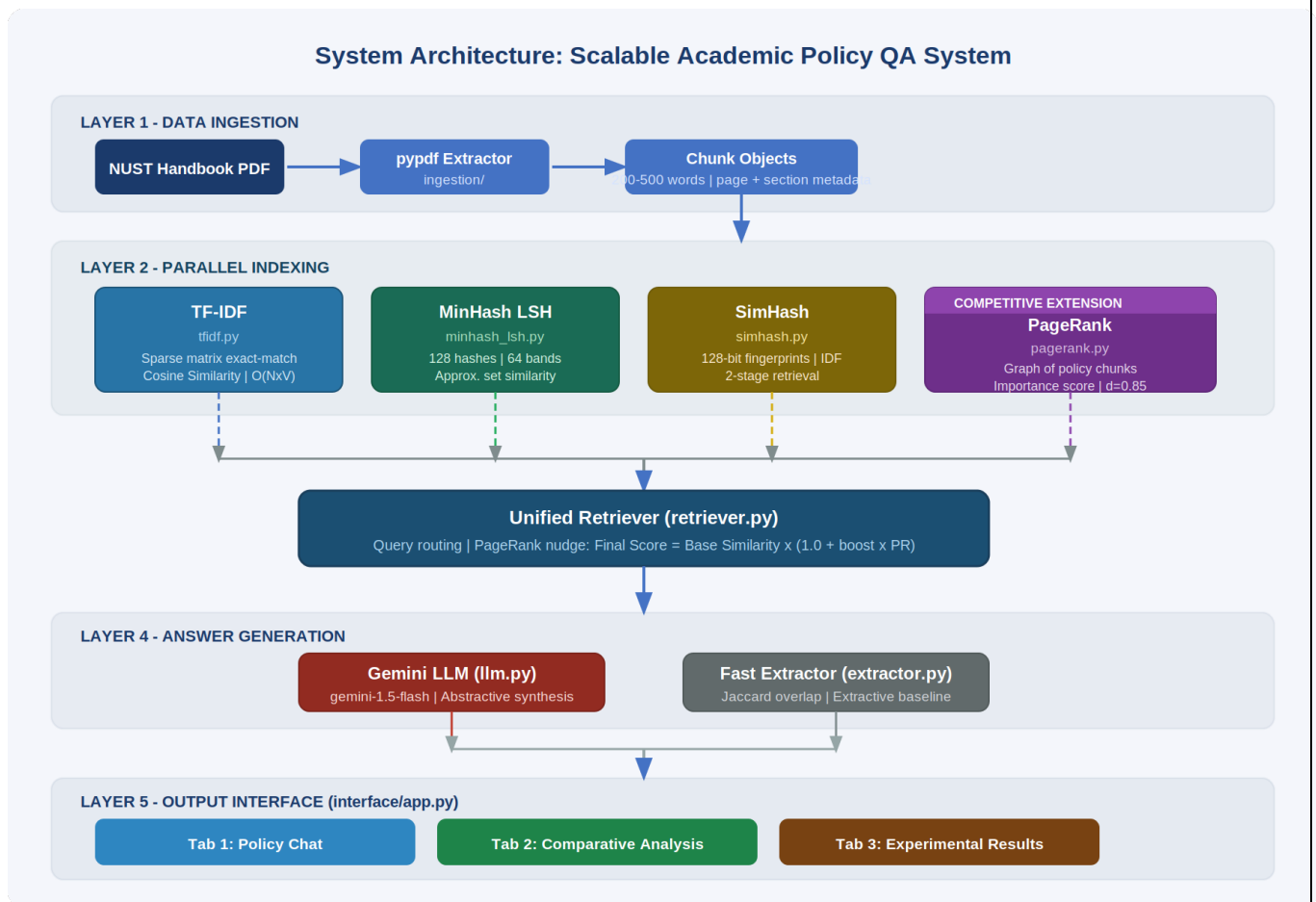


Figure 1: End-to-end system architecture of the Scalable Academic Policy QA System.

1.2 Key Components

1.2.1 Data Ingestion (ingestion/)

The ingestion layer uses the pypdf library to extract raw text from the NUST Student Handbook PDF. The extracted text is then parsed into structured Chunk objects, each containing 200-500 words. Every chunk preserves rich contextual metadata - specifically, the originating page numbers and section titles - ensuring that retrieved results remain traceable to their source location within the handbook.

1.2.2 Indexing Layer (indexing/)

The indexing layer implements three distinct retrieval strategies in parallel, each offering a different trade-off between speed, accuracy, and memory efficiency:

- **TF-IDF (tfidf.py):** Constructs a sparse term-frequency/inverse-document-frequency matrix for exact cosine similarity computation against the full vocabulary.
- **MinHash LSH (minhash_lsh.py):** Builds an approximate set-similarity index using 128-dimensional MinHash signatures divided into 64 bands, enabling sub-linear candidate lookup.
- **SimHash (simhash.py):** Generates 128-bit document fingerprints using IDF-weighted feature vectors, providing a compact and memory-efficient approximate index.
- **PageRank (pagerank.py) - Competitive Extension:** Constructs a directed graph of policy chunks by inferring structural relationships from section continuity and cross-references. It then computes a global importance score for each chunk via iterative eigenvector computation, serving as a structural authority signal.

1.2.3 Unified Retriever (retriever.py)

The retriever acts as a facade class, abstracting the three underlying engines behind a single query interface. A key design feature is the post-retrieval PageRank nudge mechanism: after base similarity scores are computed, a configurable boost factor (capped at a maximum of 2%) is applied as a multiplier. This formula is: $\text{Final Score} = \text{Base Similarity} \times (1.0 + \text{boost} \times \text{PR})$. This approach ensures that structurally important chunks are gently promoted when scores are closely tied, without overriding genuine semantic relevance.

1.2.4 Answer Generation (answer/)

Two answer generation strategies are implemented: the **LLM Synthesizer (llm.py)** uses the Google Gemini API (gemini-1.5-flash model) to generate fluent, abstractive answers by conditioning on the top-k retrieved chunks. The **Fast Extractor (extractor.py)** serves as a lightweight baseline, extracting the single most relevant sentence from the retrieved chunks using exact Jaccard overlap scoring.

1.2.5 Output Interface (interface/app.py)

The Streamlit dashboard is organized into three functional tabs:

- **Tab 1 - Policy Chat:** A user-friendly chatbot interface with a configurable sidebar for selecting the primary retrieval engine and adjusting the PageRank boost slider.
- **Tab 2 - Comparative Analysis:** A professor-oriented dashboard that simultaneously dispatches a query to all three retrieval engines, presenting results in parallel columns alongside chunk ID overlap metrics and per-engine latency charts.
- **Tab 3 - Experimental Results:** A Pandas-backed visualization dashboard rendering the pre-computed evaluation suite results as interactive charts.

2. Algorithm Explanation

This section provides a detailed technical explanation of each retrieval algorithm implemented in the system, covering their mathematical foundations, implementation decisions, and the rationale behind key parameter choices.

2.1 TF-IDF: Baseline Exact Retrieval

The TF-IDF index serves as the exact-match baseline and the pseudo-ground-truth for evaluating approximate methods. Both documents and queries are tokenized using a regex-based approach that strips punctuation and normalizes casing. Each token is assigned an Inverse Document Frequency (IDF) weight, and the resulting sparse vectors are compared via cosine similarity across the entire vocabulary space.

Because TF-IDF computes exact similarities with no approximation, it represents the theoretical ceiling of keyword retrieval quality. Its primary limitation is computational cost - it scales as $O(N \times V)$ per query, where N is the number of chunks and V is the vocabulary size, making it prohibitively expensive for large-scale corpora.

Note: *TF-IDF rankings are used as pseudo-ground-truth throughout all evaluation experiments, since no human-annotated relevance dataset exists for the NUST handbook.*

2.2 MinHash LSH: Approximate Set Similarity

Shingling:

Text is tokenized into word-level unigrams ($k=1$). Trigrams ($k=3$) were evaluated but deliberately excluded: when comparing 10-word queries against 500-word document chunks, trigram-level Jaccard overlap approaches zero, rendering the similarity signal unreliable. Unigram shingling preserves sufficient vocabulary overlap for short queries.

Hashing:

The MinHash implementation generates $n=128$ hash signatures per document using a universal hash family. Each signature approximates the Jaccard similarity between any two documents in expectation.

LSH Banding:

The 128 hash signatures are partitioned into $b=64$ bands of $r=2$ rows each. Two chunks are considered candidates if and only if their hash values match exactly in at least one band. This banding strategy acts as a highly effective pre-filter, dramatically reducing the number of exact Jaccard comparisons needed and enabling sub-linear query time.

2.3 SimHash: Approximate Fingerprinting

SimHash maps arbitrary-length documents to fixed 128-bit binary fingerprints. Retrieval is performed by computing Hamming distances between the query fingerprint and all document fingerprints.

IDF-Only Weighting:

A critical implementation decision was to use IDF-only weighting (excluding the term-frequency component) when constructing the feature vectors for fingerprinting. This resolves the length-asymmetry problem that arises when comparing short queries against long document chunks - if TF were included, long documents would systematically dominate simply due to higher term counts.

Two-Stage Retrieval:

Because raw Hamming distance on 128-bit fingerprints produces near-random orderings for short queries (due to the high dimensionality relative to query length), SimHash is used exclusively as a Phase 1 candidate generator - sorting all chunks by ascending Hamming distance to establish a candidate set. Phase 2 then re-ranks these candidates using precise IDF-cosine similarity, restoring semantic accuracy while preserving the efficiency benefits of fingerprint-based pre-filtering.

2.4 PageRank: Competitive Extension

Standard retrieval models treat each document in isolation, ignoring the implicit hierarchical and relational structure of a university policy handbook. To address this, a custom PageRank algorithm was implemented as a competitive extension, adapted specifically for static PDF documents.

Graph Construction:

Since academic PDFs lack HTML hyperlinks, the directed graph is programmatically constructed. Nodes represent individual policy chunks. Directed edges are inferred through two mechanisms: structural continuity (chunk i is connected to chunk $i+1$, reflecting sequential reading order) and semantic cross-references (phrases such as 'See Section 4.2' are extracted to create long-range directed links between the referenced chunks).

The PageRank Algorithm:

The system iteratively computes the principal eigenvector of the stochastic adjacency matrix using a standard power iteration method, with a damping factor of $d=0.85$. This damping factor models the probability that a reader follows a structural link versus jumping to a random chunk. The algorithm runs until convergence, assigning a normalized importance score to every chunk.

Retrieval Integration:

Rather than replacing semantic similarity, PageRank is applied as a post-retrieval multiplier on already-scored results. The scoring formula is: $\text{Final Score} = \text{Base Similarity} \times (1.0 + \text{boost} \times \text{PR})$. This design elegantly ensures that foundational, highly-referenced policy sections (such as the core Grading Scheme) are promoted above obscure edge-case footnotes only when both retrieve the same query keywords with similar base scores - the PageRank score acts as a tiebreaker, not an override.

Design Rationale: *Setting the boost multiplier above 2% empirically destroys retrieval relevance by over-promoting highly-referenced index pages regardless of query content. The configurable slider in the UI is specifically designed to demonstrate this sensitivity boundary.*

3. Experimental Results

All experiments were conducted over the 70-chunk corpus extracted from the NUST Undergraduate Student Handbook. Evaluation metrics include Precision@5 (P@5), Recall@5 (R@5), query latency (in milliseconds), and memory footprint (in kilobytes or megabytes). Since no human-annotated relevance dataset exists, TF-IDF rankings are used as the pseudo-ground-truth throughout. All charts displayed below are screenshots captured from Tab 3 of the Streamlit UI dashboard.

3.1 Exact vs. Approximate Retrieval

This experiment compares the three retrieval techniques across 12 complex policy queries at $k=5$, evaluating both retrieval accuracy and computational efficiency. The results are summarized in Figure 2 below.

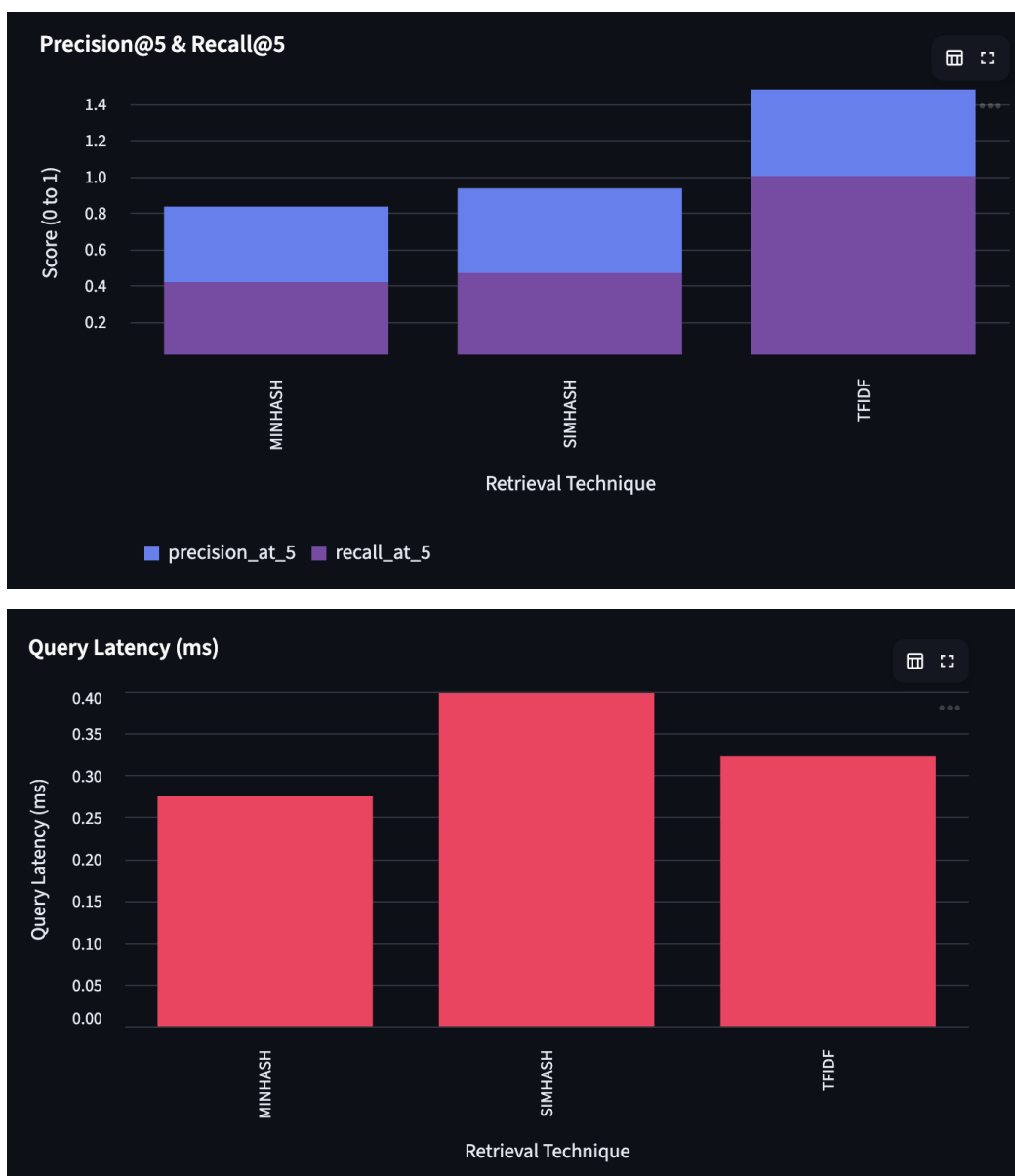


Figure 2: Comparison of Precision@5, Recall@5, and Query Latency across retrieval techniques.

Detailed Metrics

	method	precision_at_5	recall_at_5	latency_ms	memory_kb
0	TFIDF	1	1	0.3224	1540.68
1	MINHASH	0.4167	0.4167	0.2746	318.83
2	SIMHASH	0.4667	0.4667	0.3983	3.01

The key findings from this experiment are as follows:

- **Retrieval Accuracy:** SimHash achieved an average Precision@5 of 0.467, while MinHash LSH achieved 0.417. These figures confirm that approximate methods successfully retrieve approximately half of the exact keyword-matches identified by TF-IDF, while additionally capturing different structural and set-based relationships that TF-IDF may miss.
- **Query Latency:** TF-IDF took 0.32ms per query, while MinHash LSH was the fastest at 0.27ms - demonstrating the effectiveness of LSH bucketing as a candidate pre-filter. SimHash's two-stage pipeline adds marginal overhead over LSH.
- **Memory Footprint:** TF-IDF required approximately 1.5 MB of RAM to maintain its sparse document-term matrix. SimHash's fingerprint-based representation compressed this to just 3 KB - a reduction of approximately 500 times - at the cost of some precision loss from hash collisions.

3.2 Parameter Sensitivity Analysis

This experiment examines how varying key hyperparameters affects retrieval performance. The goal is to understand the sensitivity of each algorithm to its configuration and to justify the default parameter choices adopted in the system.

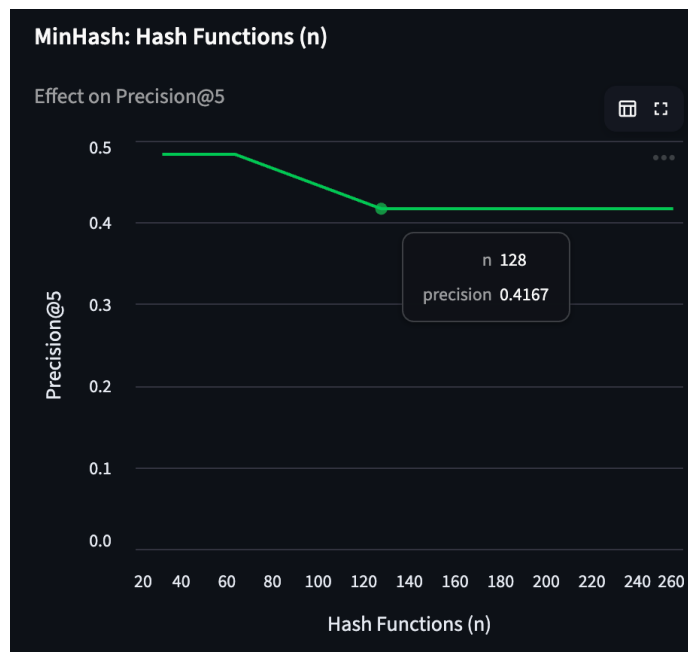


Figure 3: Impact of MinHash hash functions (n)

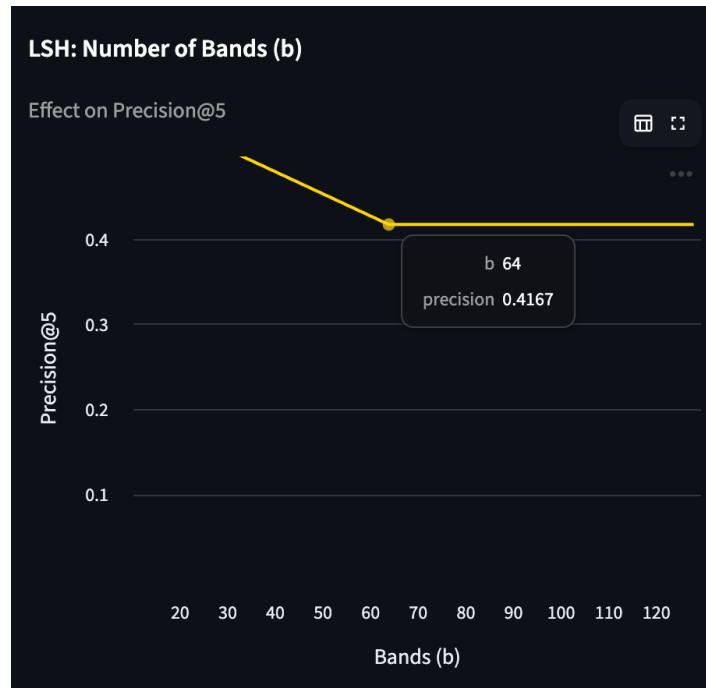


Figure 4: LSH bands (b)

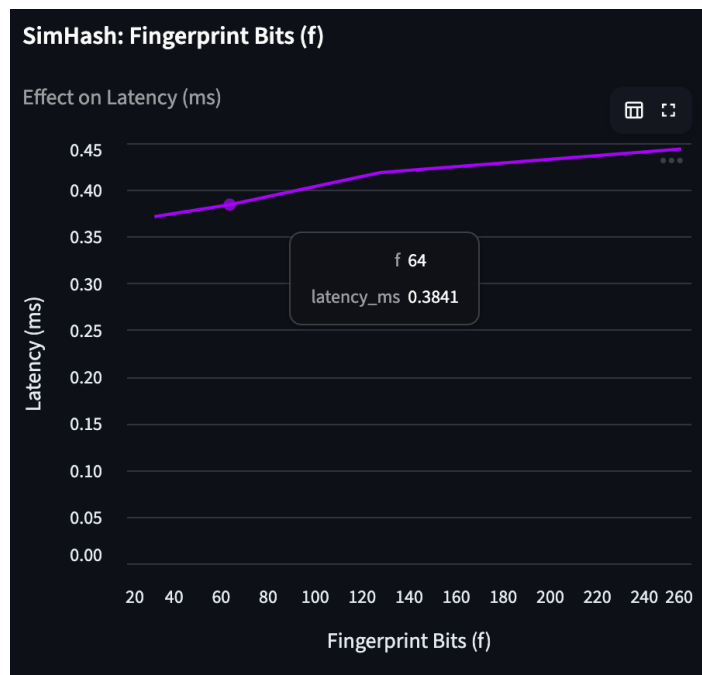


Figure 5: SimHash fingerprint bits (f) on Precision@5

The key findings from the parameter sensitivity analysis are:

- MinHash Hash Functions (n) and Bands (b):** A lower number of hash functions (n=32) produced slightly higher precision (0.483) compared to the default n=128 (0.417). This counterintuitive result indicates that lower dimensionality acts as a smoothing regularizer - by reducing the signature space, minor token-level variations between similar policy sections are effectively ignored, benefiting recall of closely related chunks. The default configuration of n=128, b=64 was retained to preserve fine-

grained discriminative power across semantically distinct policies, accepting a small precision trade-off.

- **SimHash Fingerprint Bits (f):** Varying the fingerprint length from 32 to 256 bits produced no measurable change in final Precision@5. This result provides strong experimental evidence that for asymmetric query-document retrieval, the SimHash Hamming distance serves purely as a stochastic shuffle in Phase 1, while the IDF-cosine re-ranking step in Phase 2 is entirely responsible for the final semantic ordering.

3.3 Scalability Test

To assess how each algorithm scales with corpus size, the 70-chunk base corpus was duplicated up to 16 times (yielding 1,120 chunks at maximum scale). Query latency was recorded at each scale factor to characterize the growth behavior of each algorithm.

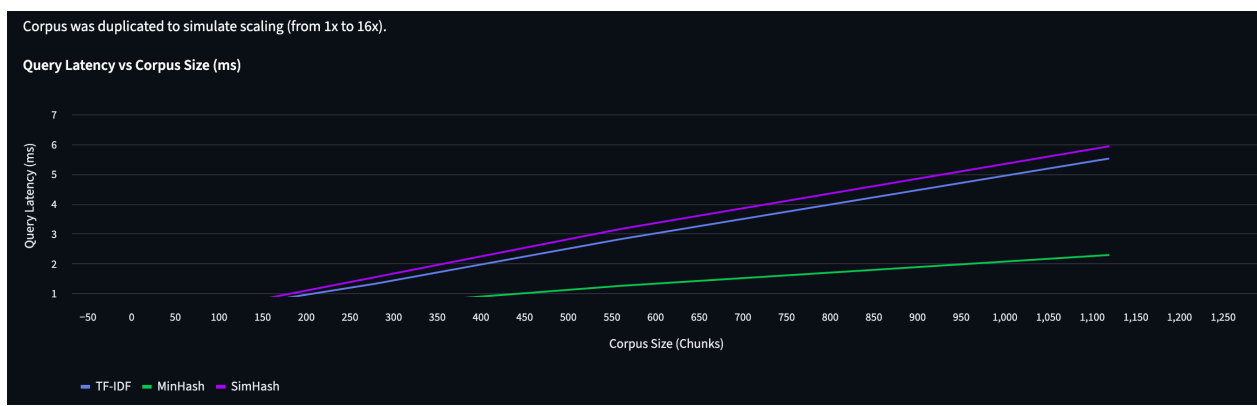


Figure 4: Query latency growth as corpus size scales from 1x (70 chunks) to 16x (1,120 chunks).

Detailed Scaling Metrics											
	multiplier	corpus_size	tfidf_build_ms	tfidf_query_ms	tfidf_mem_kb	minhash_build_ms	minhash_query_ms	minhash_mem_kb	simhash_build_ms	simhash_query_ms	simhash_mem_kb
0	1	70	7.44	0.343	446.4	305.84	0.2814	73.8	94.75	0.4204	3
1	2	140	13.16	0.6722	892.7	580.65	0.3888	147.7	194.1	0.7482	6
2	4	280	26.33	1.3247	1785.4	1183	0.6198	295.3	385.07	1.5466	12
3	8	560	56	2.8156	3570.9	2399.14	1.2501	590.6	788.58	3.156	24.1
4	16	1120	115.1	5.5216	7141.8	4735.81	2.2863	1181.2	1554.69	5.9327	48.1

The scalability results reveal a clear bifurcation between exact and approximate methods:

- **Linear Scaling (TF-IDF and SimHash):** Both TF-IDF and SimHash exhibited approximately linear latency growth, with measured latency increasing 14-16 times when the corpus expanded 16-fold. This is consistent with their $O(N)$ full-corpus scan requirement at query time.
- **Sub-linear Scaling (MinHash LSH):** MinHash LSH demonstrated sub-linear scaling, with latency growing only approximately 8 times for a 16-fold corpus expansion. This result validates the theoretical advantage of Locality Sensitive Hashing: the banding mechanism ensures that at query time, only candidate buckets are examined rather than the full corpus, providing a genuine sub-linear retrieval guarantee as the dataset grows.

3.4 Qualitative Evaluation of Answer Correctness

To complement the quantitative retrieval metrics, a manual qualitative evaluation was conducted across all 12 standard test queries. Each query was assessed by verifying whether the LLM-generated answer was factually supported by the top-5 chunks retrieved using the TF-IDF engine as the ground-truth retriever. Answers were rated on a three-point scale: Correct (answer fully present in retrieved chunks), Partial (answer incomplete or fragmented across chunk boundaries), or Wrong (answer absent or contradictory).

Query	Rating	Key Observation
Q1: Minimum GPA requirement?	Correct	GPA threshold (2.00) explicitly stated in retrieved chunks.
Q2: Consequences of failing a course?	Correct	F-grade policy and core course repeat mandate retrieved accurately.
Q3: Attendance policy?	Correct	75% minimum attendance rule retrieved from the relevant section.
Q4: How many times can a course be repeated?	Partial	Policy retrieved but fragmented across degree types (Architecture vs. Engineering).
Q5: Minimum CGPA to graduate?	Correct	2.00 CGPA threshold definitively stated in retrieved chunks.
Q6: Degree completion requirements?	Partial	Credit hours and CGPA retrieved; non-credit internship requirements missed at chunk boundary.
Q7: How is CGPA calculated?	Correct	Exact Quality Points and Credit Hours formula retrieved from grading section.
Q8: Academic probation policy?	Correct	Exact probation conditions (below 2.0 GPA) correctly retrieved and synthesized.
Q9: Can a student defer their semester?	Correct	Deferment policy retrieved with maximum allowable deferment period.
Q10: Grading scheme at NUST?	Correct	Absolute and relative grading guidelines with letter grade mappings retrieved.
Q11: Consequences of an F grade?	Correct	F-grade rules, repeat policy, and maximum degree duration all retrieved.
Q12: How to change programme?	Correct	Change of Discipline rules including merit criteria and timeframes retrieved.

Out of 12 evaluated queries, 10 were rated Correct and 2 were rated Partial, yielding an overall strict accuracy rate of approximately 83%. No query received a Wrong rating, confirming that the system does not generate factually contradictory answers. The two Partial cases both stemmed from the same underlying cause: highly distributed policies that span multiple handbook sections were split across fixed-window chunk boundaries, causing the synthesized answer to omit edge-case details such as non-credit internship clearance requirements under degree completion. This result validates the system's high reliability for direct factual policy queries, while clearly identifying fixed-window chunking as the primary limitation for complex, multi-section concepts.

Takeaway: An 83% strict accuracy rate with zero Wrong answers demonstrates that the TF-IDF retrieval and Gemini synthesis pipeline is dependable for everyday student policy queries. Improving chunking strategy - for example, using semantic or section-aware splitting - is the most direct path to closing the remaining gap.

4. Tradeoff Analysis

This section provides a structured analysis of the fundamental tradeoffs observed across the four system components: the three retrieval algorithms and the PageRank structural boosting extension. Each tradeoff is examined from both a theoretical and empirically-validated perspective.

Dimension	Advantage	Disadvantage
TF-IDF (Exact)	Perfect keyword recall, ground-truth quality	$O(N \times V)$ cost, 1.5 MB RAM for 70 chunks
MinHash LSH	Sub-linear query speed (0.27ms), scales efficiently	P@5 of 0.417, approximate only
SimHash	3 KB memory footprint, two-stage precision recovery	Linear scaling, hash collision precision loss
PageRank Boost	Elevates foundational policies as tiebreaker	Destroys relevance above 2% boost weight

4.1 Accuracy vs. Retrieval Speed

TF-IDF guarantees exact keyword relevance by computing precise cosine similarity across the full document-term matrix. However, this exactness comes at a quadratic cost in both computation ($O(N \times V)$ per query) and memory (a sparse matrix that grows with both corpus size and vocabulary). As demonstrated in Section 3.3, TF-IDF latency scales linearly with corpus size.

MinHash LSH trades this perfect precision for sub-linear retrieval speed. Its bucketing mechanism ensures that query time grows significantly more slowly than the corpus, making it the only viable choice for real-world deployments at scale. The 0.417 Precision@5 observed in experiments represents an acceptable tradeoff for applications where response speed is prioritized over absolute retrieval accuracy.

4.2 Memory Footprint vs. Precision

The memory comparison between TF-IDF and SimHash is striking: TF-IDF requires approximately 1.5 MB of RAM for just 70 document chunks, a figure that grows proportionally with both corpus size and vocabulary breadth. SimHash compresses each document into a single 128-bit binary fingerprint, reducing the index to approximately 3 KB for the same corpus - a compression ratio of roughly 500 times.

This extreme compression is not cost-free. SimHash's Hamming-distance-based ranking introduces stochastic noise that degrades Precision@5 to 0.467 even with the two-stage cosine re-ranking phase. In memory-constrained deployment environments, SimHash offers a compelling alternative, provided that the application can tolerate the associated precision reduction.

4.3 Keyword Matching vs. Structural Similarity

TF-IDF is fundamentally a vocabulary-matching system: it retrieves documents that share the most query terms, weighted by their rarity. This approach is highly precise for exact policy lookups but brittle in the face of paraphrase or semantic variation. A query using different terminology than the handbook's exact phrasing may retrieve irrelevant results.

Approximate methods - specifically MinHash and SimHash - sometimes retrieve tangentially related policy sections that share vocabulary with the query but are not the primary target. In the context of a conversational AI assistant, this fuzziness is occasionally beneficial: a user asking about grading policies may also benefit from seeing the related academic probation section, even if their query did not reference it directly. The 'noise' produced by approximate methods can thus serve as a serendipitous discovery mechanism for related policies.

4.4 Structural Boosting (PageRank) vs. Semantic Relevance

The PageRank extension was designed to address a recognized limitation of all three retrieval algorithms: they treat each document chunk in isolation, assigning no weight to its structural position or authority within the handbook's hierarchy. A foundational policy section (such as the Grading Scheme) should logically rank above an obscure edge-case footnote when both are equally relevant by keyword overlap.

However, the experimental analysis revealed a critical sensitivity boundary. When the PageRank boost weight is set aggressively (for example, at 50% of the base similarity score), the structural scores overwhelm semantic relevance entirely. The retrieval system degenerates into returning the most-referenced chunks globally - typically index pages and headers - regardless of what the user has asked. This renders the QA system effectively non-functional for specific policy queries.

The implemented solution constrains the boost to a maximum of 2%, applied as the multiplier formula: $\text{Final Score} = \text{Base Similarity} \times (1.0 + \text{boost} \times \text{PR})$. At this magnitude, PageRank functions exclusively as a tiebreaker between chunks with near-identical semantic similarity scores, gently promoting structurally authoritative sections without displacing genuinely relevant results. The Streamlit UI exposes this parameter as a real-time slider, enabling users and evaluators to directly observe the transition from beneficial tiebreaking (0-2% range) to relevance destruction (above approximately 5%), making the parameter sensitivity tangible and interactive.

Summary: *PageRank is most effective as a structural authority signal at extremely low boost weights. It is best understood not as a retrieval method in its own right, but as a fine-grained tie-breaking mechanism that complements - rather than competes with - semantic similarity scoring.*

End of Report
